

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

Learning from Comparison Data

M.Tech Project: Phase I Report

Submitted by

Nitesh Kumar Shah

ID25M806

Under the Supervision of

Prof. Suryanarayana Sankagiri

Department of Data Science and Artificial Intelligence

Indian Institute of Technology Madras

Chennai – 600036

November 2026

Abstract

Classical choice models such as Bradley terry Luce(BTL) model assumes that the relative preference between two alternatives remains unchanged when other alternatives are added or removed. This property is called Independence of Irrelevant Alternatives (IIA). However, the available alternatives can influence a person's choice, which leads to the violations of IIA. Motivated by Discovering Context Effects from Raw Choice Data by Seshadri et al., this project studies how such context effects can be represented and learned from observed choice data.

This project begins with the Bradley–Terry model to understand parameter recovery and estimation from pairwise comparison data by generating synthetic data, we studied parameter recovery, the effects of comparison data, the existence of a finite maximum likelihood estimates, and the behaviour of different optimization methods. We then studied and implemented the Context-Dependent Model (CDM), which allows an item's utility to depend on the other items in the choice set.

The experiments conducted so far helped us to examine how the CDM recovery changes with the number of observations and items. We also fit Bradley–Terry and CDM to data generated by each other and compare their predictive performance on unseen choices. Finally, we investigated the low-rank CDM as a way to reduce the number of model parameters and study its computational behaviour. The results illustrates that the context-dependent model improves prediction in the synthetically generated choice data.

The code and experiment notebooks are available at:

https://github.com/rbhdsks/Learning-from-Comparison-Data-Master-Thesis-Project-

Keywords: Preference learning; Bradley–Terry model; context-dependent choice; independence of irrelevant alternatives; maximum likelihood estimation; low-rank models.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

Let us try to imagine that someone has to get from IIT Madras to the airport, and they have 4 real choices like Car, Bike, taxi, bus.. And so, based on the cost, comfort, speed they have some natural preference ranking among this 4, let's say they choose or pick

$$\text{Car} > \text{Taxi} > \text{Bike} > \text{Bus}$$

Now, suppose we add a 5th, a clear irrelevant option to the list which would almost take 3 hours and nobody would seriously choose this. Common sense tells us that adding 'walking' doesn't really change the feel about the car v/s the taxi v/s the bike, as it is the irrelevant alternative, and their **relative preference** between all these 4 alternative would not change at all.

So this idea is called independence of irrelevant alternatives and the core behind the classical model of choice.

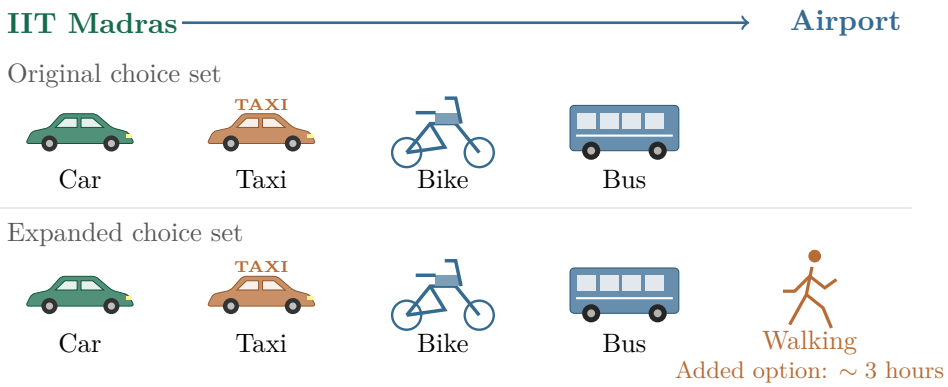


Figure 1: The commute choices from IIT Madras to the airport. Under the IIA assumption, adding walking leaves the relative odds between the original alternatives unchanged. (Here, the three-hour walking time is part of the illustrative example.)

But the core idea is that maybe human brains may not be registering the preference in the same way of that of multinomial logits. As there is a strong experimental evidence proves that it would shift the attractiveness of things in the way that how attractive the existing options really looks → this is actually called violations of independence of irrelevant alternatives.[**?**]

2 Background and Choice Models

2.1 Choice Data and Choice Sets

Let $\mathcal{X}$ denote the universal set of alternatives. A choice set is a subset of $\mathcal{X}$ containing the alternatives available to a decision-maker at a particular time. As a separate example, consider travel from IIT Madras to the airport. The universal set of transport alternatives is $\mathcal{X} = \{\text{Car}, \text{Taxi}, \text{Bike}, \text{Bus}, \text{Walking}\}$. The walking alternative remains a option of the universal set even when it does not appear in a particular choice set. All the smaller choice sets are constructed from the same fixed universal set. Let $A, B \subseteq \mathcal{X}$ be two choice sets. Let x and y be alternatives belonging to the intersection $A \cap B$.

2.2 Independence of Irrelevant Alternatives

For a universal set $\mathcal{X}$ and choice sets $A, B \subseteq \mathcal{X}$, the Independence of Irrelevant Alternatives property for two alternatives x and y can be written as

$$\frac{P(x \mid A)}{P(y \mid A)} = \frac{P(x \mid B)}{P(y \mid B)}, \qquad x, y \in A \cap B. \tag{1}$$

The equation states that if x and y are present in both choice sets, then their relative odds should not change merely because other alternatives have been added or removed.

2.3 Multinomial Logit Model

The classical multinomial logit model assigns a score $v(x)$ to every alternative x. The probability of selecting x from choice set C is

$$P(x \mid C) = \frac{\exp(v(x))}{\sum_{y \in C} \exp(v(y))}. \tag{2}$$

When the choice set contains two alternatives, this gives the Bradley–Terry model.

2.4 Bradley–Terry Model and Maximum Likelihood

2.4.1 Pairwise Comparison Setting

To understand the underlying mechanics of a model for choice data, we consider observed pairwise comparisons and assume a probabilistic model. The Bradley–Terry model, which is originally developed for analysing paired comparisons, provides a probabilistic framework for estimating the relative strengths of competing items [**?**].

The goal is to estimate unknown parameters that provide a compressed representation of the observed comparison data. For items i and j, with parameters θ_i and θ_j, the Bradley–Terry model defines

$$P(i \succ j) = \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)}. \tag{3}$$

Dividing the numerator and denominator by $\exp(\theta_i)$ gives $P(i \succ j) = \frac{1}{1+\exp(-(\theta_i - \theta_j))}$. Therefore, the probability depends only on the parameter difference $\theta_i - \theta_j$.

2.4.2 Parameter Identifiability and Centring

The Bradley–Terry probability depends only on the difference between two parameters as shown in **??**. Consequently, adding the same constant c to every parameter does not change any pairwise comparison probability.

Therefore, the parameter vectors

$$(\theta_1, \ldots, \theta_n) \qquad \text{and} \qquad (\theta_1 + c, \ldots, \theta_n + c)$$

represent the same Bradley–Terry model. The individual parameter values are consequently not identifiable without an additional constraint.

We remove this additive ambiguity using the zero-sum constraint

$$\sum_{i=1}^{n} \theta_i = 0. \tag{4}$$

For any parameter vector $\boldsymbol{\theta}$, a centred version can be obtained by subtracting its mean:

$$\theta_i^{\text{centred}} = \theta_i - \frac{1}{n} \sum_{j=1}^{n} \theta_j. \tag{5}$$

This transformation preserves every parameter difference and therefore does not change the pairwise comparison probabilities. In the numerical experiments, both the true and estimated parameter vectors are centred before their recovery error is calculated.

Shift Invariance and the Zero-Sum Constraint The same additive ambiguity occurs in MNL: adding a common constant to all scores leaves the choice probabilities unchanged. Centring removes this ambiguity by selecting scores whose sum is zero.

Consider two alternatives, car and taxi, with scores $v(C) = 2.5$ and $v(T) = -0.9$. Their mean is 0.8. Subtracting this mean gives $v(C) = 2.5 - 0.8 = 1.7$ and $v(T) = -0.9 - 0.8 = -1.7$. The centred scores sum to zero, while their difference remains 3.4. The probability of choosing car is approximately 0.968 before and after centring. Thus, adding the same constant to every score does not change the choice probabilities. The zero-sum constraint selects one representative from all score vectors that produce the same probabilities.

2.4.3 Maximum Likelihood Estimation

The model parameters can be estimated from observed pairwise outcomes using maximum likelihood. Likelihood-based estimation and computational algorithms for extensions of the Bradley–Terry model have been studied in the literature [**?**].

Suppose that the observed dataset contains m pairwise comparisons: $\mathcal{D} = \{(w_t, l_t)\}_{t=1}^{m}$, where w_t and l_t denote the winner and loser of comparison t, respectively.

Conditional on the compared pairs, the observed comparison outcomes are modelled as independent. For comparison t, let $p_t = P(w_t \succ l_t)$, using the Bradley–Terry probability in **??**.

The likelihood of the complete dataset is therefore

$$L(\boldsymbol{\theta}; \mathcal{D}) = \prod_{t=1}^{m} \frac{\exp(\theta_{w_t})}{\exp(\theta_{w_t}) + \exp(\theta_{l_t})}. \tag{6}$$

Taking the logarithm gives the log-likelihood $\ell(\boldsymbol{\theta};\mathcal{D}) = \log L(\boldsymbol{\theta};\mathcal{D})$. Maximizing the log-likelihood is equivalent to minimizing its negative. In the implementation, we use the average negative log-likelihood

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m}\ell(\boldsymbol{\theta};\mathcal{D}) = \frac{1}{m}\sum_{t=1}^{m}\log\left(1+\exp(\theta_{l_t}-\theta_{w_t})\right). \tag{7}$$

Using the average rather than the total negative log-likelihood makes the reported loss values comparable across datasets containing different numbers of comparisons.

The maximum likelihood estimate is written as

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \qquad \text{subject to} \qquad \sum_{i=1}^{n}\theta_i = 0. \tag{8}$$

2.4.4 Gradient-Based Estimation

The average negative log-likelihood in **??** is minimized using gradient descent. For one observed winner–loser pair (w_t, l_t), the loss contribution is $\mathcal{L}_t = \log(1+\exp(\theta_{l_t}-\theta_{w_t}))$, with $p_t = P(w_t \succ l_t)$.

The derivative with respect to the winner parameter is $\partial\mathcal{L}_t/\partial\theta_{w_t} = p_t - 1$, while the derivative with respect to the loser parameter is $\partial\mathcal{L}_t/\partial\theta_{l_t} = 1 - p_t$.

Therefore, when the observed winner is assigned a small winning probability, the magnitude of the update is larger. When the model already assigns the observed winner a probability close to one, the magnitude of the update is smaller.

The gradient of the average negative log-likelihood with respect to an arbitrary parameter θ_k is

$$\frac{\partial\mathcal{L}}{\partial\theta_k} = \frac{1}{m}\sum_{t=1}^{m}\left[(p_t-1)\mathbb{I}\{k=w_t\} + (1-p_t)\mathbb{I}\{k=l_t\}\right], \tag{9}$$

where $\mathbb{I}\{\cdot\}$ is the indicator function.

At gradient-descent iteration s, the complete parameter vector is updated according to

$$\boldsymbol{\theta}^{(s+1)} = \boldsymbol{\theta}^{(s)} - \eta\nabla\mathcal{L}\left(\boldsymbol{\theta}^{(s)}\right), \tag{10}$$

where $\eta > 0$ is the learning rate.

After an update, the parameter vector is centred by subtracting its mean, as in **??**.

This centring step enforces the zero-sum constraint while preserving all pairwise parameter differences. The numerical experiments use this gradient-based procedure to estimate the Bradley–Terry parameters from the simulated winner–loser observations.

2.4.5 Existence and Uniqueness of the MLE

Ford's Condition For the Bradley–Terry model, the existence of a finite maximum likelihood estimate depends on the observed comparison outcomes. These outcomes can be represented using a directed win graph, in which each vertex represents an item and an edge $i \to j$ indicates that item i has defeated item j.

Ford's condition holds when this directed win graph is strongly connected. Equivalently, for every nonempty proper subset of items, there must be at least one observed win from an item inside the subset against an item outside it, and vice versa.

When Ford's condition holds, the Bradley–Terry maximum likelihood estimate is finite. After imposing the centring constraint $\sum_{i=1}^{n}\theta_i = 0$, the estimate is unique.

2.5 From Multinomial Logit to Context-Dependent Choice Models

2.5.1 Universal Logit Representation

The universal logit model provides a framework for modelling choice probabilities [**?**]. In a universal logit model, the utility of an alternative can depend on the choice set in which it appears. Instead of assigning only one fixed utility to an alternative, the model allows it to have different utilities in different contexts. The probability of selecting x from C is

$$P(x \mid C) = \frac{\exp(u(x \mid C))}{\sum_{y \in C} \exp(u(y \mid C))}. \tag{11}$$

An identification constraint can be imposed separately for each choice set: $\sum_{y \in C} u(y \mid C) = 0, \quad$ for every $C \subseteq \mathcal{X}$.

2.5.2 Decomposition of Context-Dependent Utility

Statement of Lemma 1

Lemma 2.1 (Decomposition of context-dependent utility)**.** *Related approaches also model the influence of competing alternatives on choice probabilities, including the market-share framework proposed by Batsell and Polking [**?**]. For a universal logit model, the context-dependent utility $u(x \mid C)$ can be written as a sum of terms indexed by subsets B of $C \setminus \{x\}$ [**?**]:*

$$u(x \mid C) = \sum_{B \subseteq C \setminus \{x\}} v(x \mid B). \tag{12}$$

The parameters satisfy

$$\sum_{x \notin B} v(x \mid B) = 0 \qquad \text{for every } B \subseteq \mathcal{X}. \tag{13}$$

Interpretation of the Different Orders The term $v(x)$ represents the intrinsic preference associated with alternative x. It does not depend on the other alternatives present in the choice set. The term $v(x \mid \{z\})$ represents the contextual contribution of alternative z to the utility of alternative x. Similarly, a term such as $v(x \mid \{y, z\})$ represents a higher-order interaction involving the joint presence of alternatives y and z.

Worked Example Consider the choice set $A = \{C, T, Bi\}$. For alternative C, the utility expansion is

$$u(C \mid A) = v(C) + v(C \mid \{T\}) + v(C \mid \{Bi\}) + v(C \mid \{T, Bi\}).$$

The expansions for T and Bi follow in the same way. The constraints in **??** force the terms involving both other alternatives to zero. The remaining terms consist of the intrinsic utility and the separate contribution of each other alternative. There are nine remaining parameters and four independent constraints, giving $9 - 4 = 5$ free parameters.

2.5.3 Hierarchy of Choice Models

The decomposition gives a hierarchy of choice models. A lower-order model retains simpler contextual interactions, while a higher-order model allows the utility of an alternative to depend on larger subsets of the choice set. Increasing the order makes the model more flexible, but also increases the number of parameters that must be estimated.

The Model Classes $\mathcal{M}_p$ Let $\mathcal{M}_p$ denote the model obtained by retaining utility terms up to order p and setting all higher-order terms equal to zero. This produces the nested hierarchy $\mathcal{M}_1 \subseteq \mathcal{M}_2 \subseteq \cdots \subseteq \mathcal{M}_{n-1}$. The first-order model retains only $v(x)$. The second-order model additionally retains terms of the form $v(x \mid \{z\})$. Higher-order models include terms involving larger subsets of alternatives.

Number of Free Parameters In general, the number of free parameters in the order-p model is

$$d_p = \sum_{q=1}^{p} \binom{n}{q-1}(n-q), \qquad p = 1, \ldots, n-1. \tag{14}$$

For each context subset of size $q-1$, there are $n-q+1$ possible target alternatives. Their parameters satisfy one zero-sum constraint, leaving $n-q$ free parameters. There are $\binom{n}{q-1}$ such subsets [**?**].

For the first-order model (MNL), this gives $d_1 = n-1$. For the full universal logit model, it gives $d_{n-1} = (n-2)2^{n-1}+1$. For fixed p, the parameter count grows as $O(n^p)$: the first-order model has $O(n)$ parameters, the second-order model has $O(n^2)$, and higher orders require increasingly many parameters.

For the second-order model, there are n intrinsic parameters $v(x)$ and $n(n-1)$ contextual parameters $v(x \mid \{z\})$, giving n^2 parameters before applying the constraints. There is one zero-sum constraint on the intrinsic parameters and one for each context alternative, giving $n+1$ constraints. The number of free parameters is therefore

$$d_2 = n^2 - n - 1. \tag{15}$$

2.5.4 Context-Dependent Random Utility Model

The first-order model does not allow the utility of an alternative to change when other alternatives are added to or removed from the choice set. To represent such context effects without retaining every higher-order interaction, we consider the second-order model $\mathcal{M}_2$.

Second-Order Truncation The second-order model sets all terms involving two or more contextual alternatives equal to zero. Hence,

$$u(x \mid C) = v(x) + \sum_{z \in C \setminus \{x\}} v(x \mid \{z\}). \tag{16}$$

The choice probability is obtained by substituting this utility into **??**. Here, $v(x)$ represents the intrinsic preference associated with x, while $v(x \mid \{z\})$ represents the contribution of context alternative z to the utility of x.

Reparameterisation of the Model For $x \neq z$, define the pairwise parameter $u_{xz} = v(x \mid \{z\}) - v(z)$. Substituting $v(x \mid \{z\}) = u_{xz} + v(z)$ into the second-order utility gives a term $\sum_{z \in C} v(z)$ that is common to every alternative in the choice set. A common additive term does not change the softmax probabilities, so it can be omitted. The reparameterised utility becomes

$$u(x \mid C) = \sum_{z \in C \setminus \{x\}} u_{xz}. \tag{17}$$

Thus, the Context-Dependent Model (CDM) is expressed as

$$P(x \mid C; U) = \frac{\exp\left(\sum_{z \in C \setminus \{x\}} u_{xz}\right)}{\sum_{y \in C} \exp\left(\sum_{z \in C \setminus \{y\}} u_{yz}\right)}. \tag{18}$$

Number of Parameters The reparameterised parameters can be arranged in a matrix $U \in \mathbb{R}^{n \times n}$. Its diagonal entries are set to zero, $u_{xx} = 0$, because an alternative is not treated as its own context. There are $n(n-1)$ off-diagonal parameters, with the identification constraint

$$\sum_{x \in \mathcal{X}} \sum_{z \in \mathcal{X} \setminus \{x\}} u_{xz} = 0. \tag{19}$$

This leaves $n(n-1) - 1 = n^2 - n - 1$ free parameters, agreeing with **??**. For three alternatives, the six off-diagonal parameters satisfy one zero-sum constraint, leaving five free parameters, as in the earlier worked example.

2.5.5 Low-Rank Context-Dependent Model

In the full context-dependent model, the parameter u_{xz} represents the effect of context alternative z on target alternative x. There are $n(n-1)$ off-diagonal interaction parameters for n alternatives. The low-rank model instead constructs these interactions from smaller target and context vectors.

Each alternative x has a target vector t_x and a context vector c_x, each with r components. For two distinct alternatives, their interaction is

$$u_{xz} = t_x^\top c_z, \qquad x \neq z. \tag{20}$$

The value r is selected before fitting the model. The target and context vectors are parameters learned from choice data; they are not observed features supplied in the dataset.

If the target vectors form the rows of T and the context vectors form the rows of C, their matrix product is

$$M = TC^\top, \qquad T, C \in \mathbb{R}^{n \times r}. \tag{21}$$

Rows of M correspond to targets and columns correspond to contexts. Its off-diagonal entries are the CDM interactions. An alternative is not its own context, so diagonal entries are ignored in computing choice probabilities.

The factor product M has rank at most r. Setting its diagonal to zero may change the matrix rank; therefore, the rank restriction refers to the factor product $TC^\top$.

3 Experiments and Results

3.1 Bradley–Terry Experiments

3.1.1 Parameter Recovery Using Gradient Descent

Source notebook: `01_BradleyTerry_recovery.ipynb`

Baseline Parameter Recovery The first experiment checks whether the parameters used to generate simulated Bradley–Terry comparisons can be recovered from the observed data.

We consider $n = 10$ items. The true parameter vector is constructed using equally spaced values between -1.5 and 1.5. The vector is centred so that

$$\sum_{i=1}^{n} \theta_i = 0. \tag{22}$$

A total of $m = 1000$ pairwise comparisons are generated. Each pair is sampled uniformly, and the winner is sampled according to the Bradley–Terry probability in **??**. The random seed is fixed at 20260831 to make the experiment reproducible.

The unknown parameters are estimated by minimizing the average negative log-likelihood using gradient descent. The learning rate is set to 0.1, and the optimizer is run for a maximum of 1000 iterations.

The parameter-recovery error is measured using the root mean squared error

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\widehat{\theta}_i - \theta_i\right)^2}, \tag{23}$$

where θ_i is the true parameter and $\widehat{\theta}_i$ is its estimate.



Figure 2: Comparison of the true and estimated parameters for the baseline Bradley–Terry experiment with 10 items and 1000 pairwise comparisons.

Quantity	Value
Number of items, n	10
Number of comparisons, m	1000
Random seed	20260831
Learning rate	0.1
Maximum iterations	1000
Parameter-recovery RMSE	0.1224

Table 1: Configuration and result of the baseline Bradley–Terry parameter-recovery experiment.

As shown in **??**, the estimated parameters follow the ordering and approximate magnitudes of the true parameters. The parameter-recovery RMSE for this simulated dataset is 0.1224.

This result provides a baseline example of parameter recovery. Since it is obtained from a single simulated dataset, it does not describe the variation across independent datasets. Repeated experiments are used in the next subsection to study how the recovery error changes with the number of observed comparisons.

Effect of the Number of Comparisons The second experiment studies how the number of observed pairwise comparisons affects parameter recovery. The number of items is fixed at $n = 10$, and the same centred true parameter vector used in the baseline experiment is retained.

The number of comparisons is varied over

$$m \in \{50, 100, 200, 500, 1000, 2000, 5000\}. \tag{24}$$

For each value of m, the experiment is repeated independently 20 times. A new comparison dataset is generated in every run, while the true parameters, pair-sampling method, learning rate, and maximum number of gradient-descent iterations are kept fixed. The mean parameter-recovery RMSE and its standard deviation are then computed over the 20 runs.

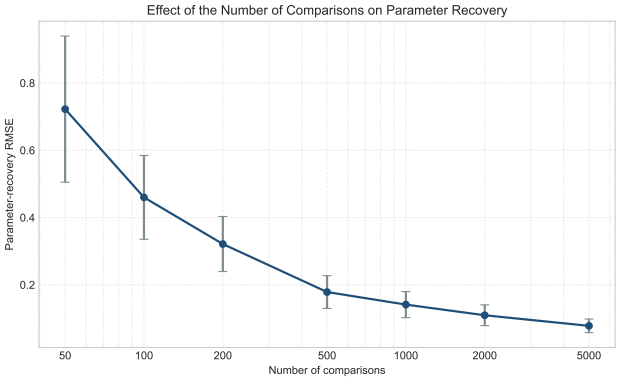


Figure 3: Mean Bradley–Terry parameter-recovery RMSE as the number of comparisons increases. The error bars show one standard deviation across 20 independent runs.

Comparisons m	Mean RMSE	Standard deviation
50	0.7225	0.2172
100	0.4600	0.1247
200	0.3213	0.0819
500	0.1786	0.0487
1000	0.1412	0.0388
2000	0.1095	0.0309
5000	0.0780	0.0203

Table 2: Parameter-recovery error for different numbers of pairwise comparisons. Each result is computed over 20 independent runs.

As shown in **????**, the mean parameter-recovery RMSE decreases as the number of observed comparisons increases. It decreases from 0.7225 with 50 comparisons to 0.0780 with 5000 comparisons.

The standard deviation also decreases from 0.2172 to 0.0203. Thus, in this experimental setting, increasing the amount of observed comparison data improves both the average recovery accuracy and the consistency of the estimates across independent datasets.

3.1.2 Bradley–Terry Recovery versus Number of Items Using Choix

Source notebook: `00_BTL_recovery_noofitems.ipynb`

This notebook studies how accurately the true parameters can be recovered from simulated comparison data. We use $n \in \{10, 20, 50, 100, 200\}$ and repeat each setting five times. The true parameters are sampled from a standard normal distribution and centred. Pairs are sampled uniformly, and parameters are fitted using the Choix library.

Fixed Total Comparisons (Left) We keep the total number of comparisons fixed at $m \in \{1000, 5000, 20000, 100000\}$ and increase the number of items. The same data budget is now shared among more items. Since every comparison involves two items, each item appears in $2m/n$ comparisons on average, so this average decreases as n increases.

Fixed Comparisons per Item (Right) We instead keep m/n fixed at 100 or 500. As more items are added, the total number of comparisons increases in the same proportion. The expected number of appearances per item, $2m/n$, therefore stays at 200 or 1,000, respectively. This checks recovery when the data budget grows with the number of items.

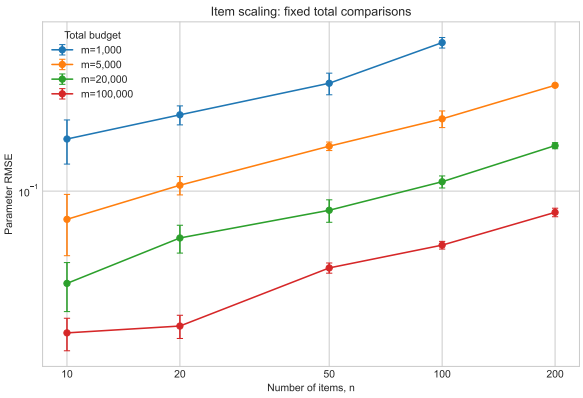


Figure 4: Parameter RMSE as the number of items increases at fixed total comparison budgets m.

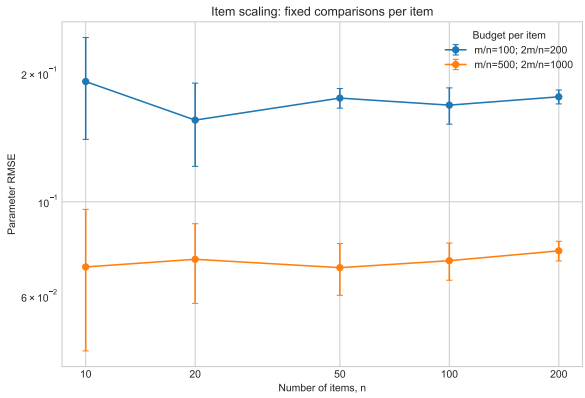


Figure 5: Parameter RMSE as the number of items increases at fixed $m/n = 100$ and $m/n = 500$.

Both graphs show the mean RMSE across converged runs, with error bars showing one standard deviation. Each setting has five attempted runs. Only runs with a finite MLE and a converged fit are included. At $n = 200$ and $m = 1000$, none of the five runs has a finite MLE, so that point is not shown.

Interpretation At fixed m, recovery generally becomes worse as the number of items increases. At fixed m/n, the error stays roughly stable, and the larger comparison budget gives lower error.

3.1.3 Comparison of Optimization Methods

Source notebook: `04_BradleyTerry_MLE_Optimizations.ipynb`

Experimental Setup The Bradley–Terry negative log-likelihood was minimized using five optimization methods: full-batch gradient descent, stochastic gradient descent (SGD), mini-batch gradient descent, gradient descent with momentum, and Adam.

Each method was implemented in two ways: a handcrafted implementation and an implementation using PyTorch. The same simulated comparison dataset and initial parameter vector were used for all methods.

A maximum budget of 10,000 epochs was used. Convergence was measured using the norm of the gradient evaluated on the complete dataset. The convergence tolerance was set to

$$\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2 \leq 10^{-4}. \tag{25}$$

For easier comparison, define the convergence ratio

$$r = \frac{\|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2}{10^{-4}}. \tag{26}$$

Thus, a run satisfies the convergence criterion when $r \leq 1$.

Final Solution Quality Table **??** summarizes the parameter-recovery error, convergence ratio, and relative runtime. Here, H and P denote the handcrafted and PyTorch implementations, respectively.

Table 3: Comparison of the optimization methods. The runtime ratio is PyTorch runtime divided by handcrafted runtime.

Optimizer	RMSE (H/P)	r (H/P)	Runtime ratio (P/H)
Full-batch GD	0.0381/0.0381	1.00/1.00	1.34
SGD	0.0393/0.0395	4.80/4.54	2.59
Mini-batch GD	0.0384/0.0387	0.90/6.31	27.82
Momentum GD	0.0404/0.0360	7.92/9.82	2.15
Adam	0.0408/0.0438	24.94/29.12	2.31

The recovery RMSE values are similar across the different methods, lying between 0.0360 and 0.0438. Therefore, all methods produced parameter estimates close to the true parameters in this experiment. However, similar recovery errors do not imply that every method satisfied the numerical convergence criterion.

Convergence Criterion Figure **??** shows the final gradient norm relative to the selected tolerance.

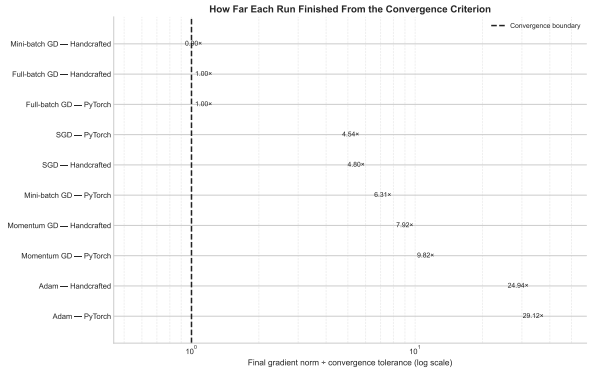


Figure 6: Final gradient norm relative to the convergence tolerance. Values at or below one satisfy the convergence criterion.

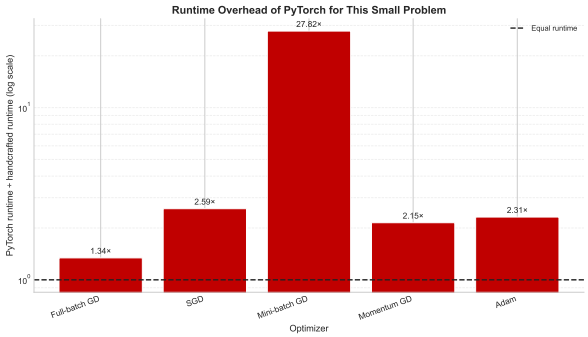


Figure 7: Ratio of PyTorch runtime to handcrafted runtime for each optimization method. A value greater than one indicates that the PyTorch implementation required more time.

Both implementations of full-batch gradient descent satisfied the convergence criterion. The handcrafted mini-batch implementation also satisfied the criterion. The remaining methods did not reach the specified tolerance within the fixed epoch budget.

In particular, stochastic methods continue to fluctuate near a solution because their updates are computed using individual observations or mini-batches. Consequently, a small recovery error may still be obtained even when the full-gradient norm remains above the strict convergence tolerance.

Handcrafted and PyTorch Runtime Figure **??** compares the runtime of the PyTorch implementation with its corresponding handcrafted implementation.

For this Bradley–Terry problem, the handcrafted implementations were faster than their PyTorch counterparts, and among all those Adam optimizer performed well. The additional runtime is consistent with the overhead introduced by tensor operations and automatic differentiation on a small CPU-based problem.

This observation is specific to the present implementation and problem size. It should not be interpreted as a general comparison of PyTorch and handcrafted optimization, since PyTorch may become advantageous for larger models, larger datasets, or GPU-based computation.

3.2 Context-Dependent Model Experiments

3.2.1 Full CDM Parameter Recovery

Source notebook: `01_CDM-CDM.ipynb`

In this experiment, we first choose known CDM parameters and use them to generate synthetic choices. We then fit a full CDM to these observations and compare the estimated parameters with the true values used to generate the data. Parameters on the diagonal are excluded because an item does not act as its own context.

Each observation records the available items and the item selected from them. A choice set contains either two or three items, and only one item is selected in each observation. For example, a two-item set records a choice between two alternatives; a three-item set records a choice among three alternatives.

We randomly divide the observations into two parts. The model is fitted using 80% of the observations, called the training data. The remaining 20%, called the test data, are kept aside to evaluate predictions on choices that were not used for fitting. For example, with 1,000 observations, 800 are used for training and 200 for testing. Parameter recovery is measured separately by comparing the fitted parameters with the known true parameters using RMSE.

A *run* means one complete repetition of generating observations, splitting the data, fitting the model, and recording the results. It is not one optimizer update. At each fixed setting, the true CDM parameters stay the same across runs, but a different random seed gives a new simulated dataset. Repeating the experiment shows how much the recovery error changes from one dataset to another.

We study two settings. First, we keep five items fixed and use 1,000, 5,000, 10,000, and 50,000 observations, with ten runs for each observation count. Second, we keep 50,000 observations fixed and use 5, 10, 20, 40, and 80 items, with five runs for each item count. Each plotted point is the mean RMSE across the runs, and the error bar shows one standard deviation.

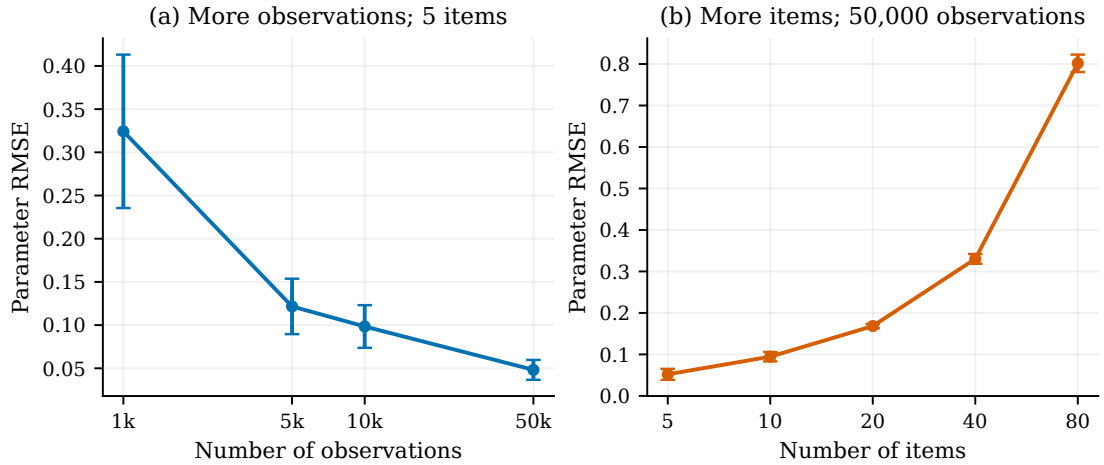


Figure 8: Full-CDM parameter recovery. (a) Observation scaling for five items, over ten runs per setting. (b) Item scaling with 50,000 total observations, over five runs per setting. Error bars show one standard deviation across runs.

Interpretation More observations improve recovery for five items. With 50,000 observations fixed, recovery becomes worse as the number of items and parameters increases.

3.2.2 CDM-Generated Choices Fitted with BTL/Luce

Source notebook: `02_CDM-BTL.ipynb`

Here, we generate choice observations using a full CDM, but fit a BTL/Luce model to them. The generating CDM can change an item's utility according to the other available items, whereas BTL/Luce uses a fixed score for each item. Fitting a model that cannot represent the generating choice pattern is called *model misspecification*. We examine whether such a gap appears in these experiments.

We use five and ten items, with choice sets containing two or three items. Each setting is repeated for ten runs. In every run, a new simulated dataset is divided into 80% training observations and 20% test observations, and BTL/Luce is fitted only to the training part.

For evaluation, we calculate the average negative log-likelihood (NLL) on the test choices. A smaller NLL means better prediction of those test choices. We compare the fitted BTL/Luce model with the true CDM parameters used for generation. The plotted gap is BTL test NLL minus true CDM test NLL, so a positive gap means BTL performs worse on those test choices.

The first comparison increases the observations from 1,000 to 50,000. The second keeps 50,000 observations fixed and changes the scale of the generating CDM parameters from zero to one.

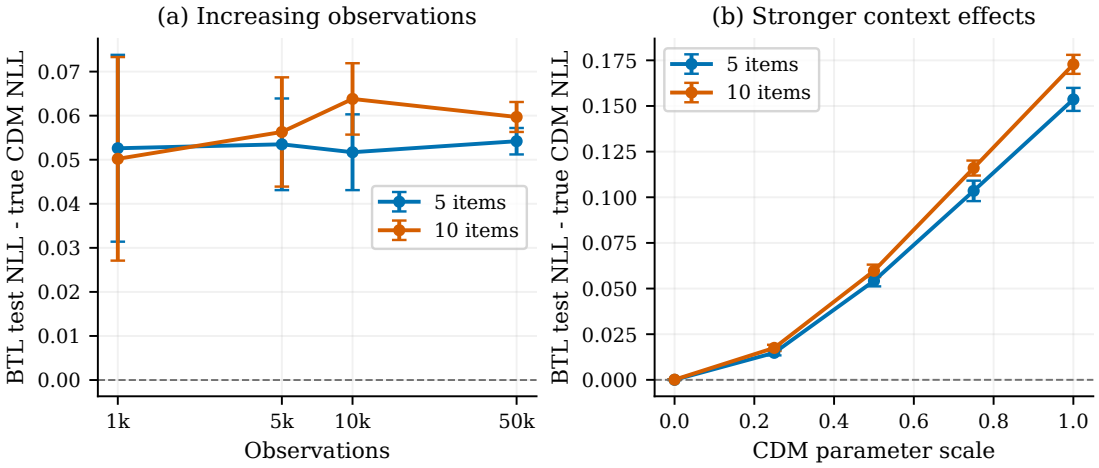


Figure 9: BTL/Luce misspecification on CDM-generated data. The gap is fitted BTL test NLL minus generating-CDM test NLL on the same choices. (a) Increasing observations. (b) Increasing interaction scale with 50,000 observations. Error bars show one standard deviation across ten runs.

Interpretation For these CDM-generated choices, more observations do not remove BTL's prediction gap. The gap increases with the CDM parameter scale, so BTL has greater difficulty representing these stronger context effects.

3.2.3 Computational Scaling of the Full and Low-Rank CDM

Source notebook: `01-FullvsLowrankResourceScaling.ipynb`

We compare full CDM with ranks one, two, and three for 250–5,000 items. Each fit uses the same 1,000 observations within a repeat and 1,000 Adam updates on the CPU, with five repeats per setting. This measures fixed-budget computational cost, not time to convergence.

The full implementation stores n^2 trainable entries, whereas the low-rank implementation stores $2nr$. These are stored entries, not identifiable free parameters. Low-rank models reduce parameter storage, but the present dense-matrix implementation still shows approximately

quadratic runtime growth. The low-rank implementation still forms the full array of CDM parameters during computation.

What RSS measures. Resident Set Size (RSS) is the amount of RAM used by the running process. It includes the model, data, temporary calculations, and Python/PyTorch memory. We sample RSS during each fit and record the highest observed value, then average these values over five repeats. The unit is MiB (2^{20} bytes). RSS therefore measures more than parameter storage, and sampling can miss a brief memory peak.

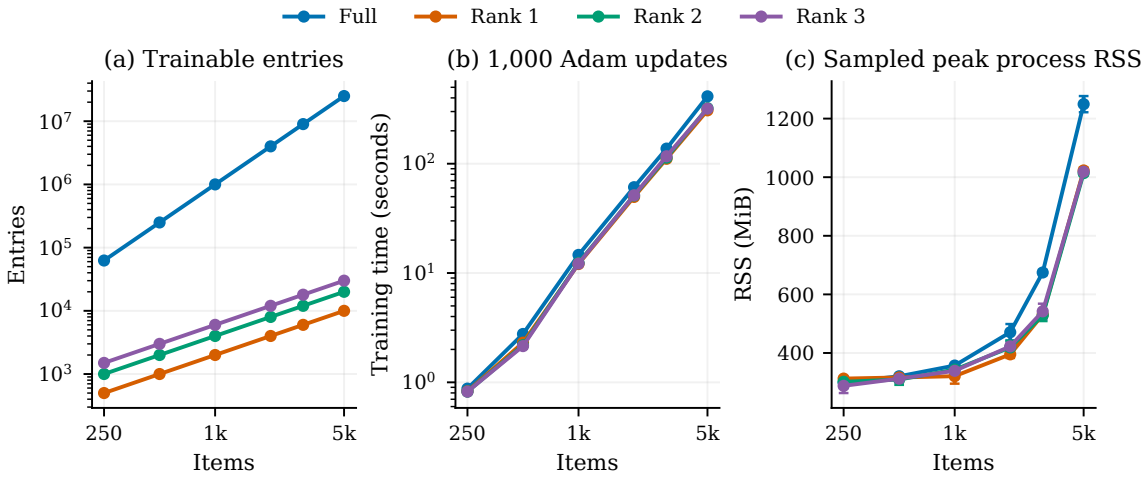


Figure 10: Computational scaling of full and low-rank CDM: (a) trainable entries, (b) time for 1,000 Adam updates, and (c) sampled peak process resident memory (RSS). Means are over five repeats; time and RSS error bars show one standard deviation. RSS measures the whole process, as explained above.

Interpretation Low-rank CDM stores far fewer parameters, but fitting time remains similar in this implementation. Its process memory is lower for large item counts, though the reduction is much smaller than the reduction in stored parameters.

3.2.4 Unified BTL, Full CDM, and Low-Rank Comparison

Source notebook: `04-Unifiednotebook.ipynb`

The models are compared on common synthetic choice sets with ten items and 10,000 observations per generating model. Both BTL-generated and CDM-generated choices include pairs and triples. The split is 70% training, 15% validation, and 15% test. Each fitted model is evaluated on the same 1,500 test choices within a generating model.

This covers BTL-to-BTL, BTL-to-CDM, CDM-to-BTL, and CDM-to-CDM fits. Low-rank fits are added to the same datasets, with ranks and restarts selected using validation NLL. The selected ranks are one for BTL data and five for CDM data.

The selected rank-five fit did not meet the gradient convergence criterion within 5,000 epochs. These are results from one synthetic setup, rather than averages over independent training datasets.

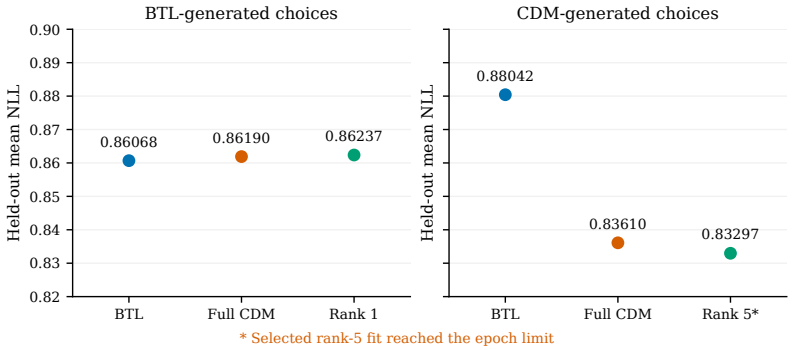


Figure 11: Held-out mean NLL (lower is better) on BTL-generated choices (left) and CDM-generated choices (right). The asterisk marks the selected rank-five fit that reached the epoch limit.

Interpretation The models perform similarly on BTL-generated choices. On CDM-generated choices, full and low-rank CDM have lower test NLL than BTL, although the selected rank-five fit has not converged.

3.2.5 Six-Item Parameter Recovery Following Seshadri et al.

Source notebook: `01_synthetic_recovery.ipynb`

Following the synthetic recovery experiment of Seshadri et al. [**?**], we study a fixed set of CDM parameters for six items, constructed from standard-normal target and context vectors. Each choice set contains between two and six items. We extend the experiment to observation counts from 1,000 to 100,000. A run is one complete repetition of generating data, fitting the model, and recording the error and fitting time. We use ten runs, each with a different random seed, while keeping the true parameters fixed. Within a run, smaller datasets are taken from the same pool of 100,000 observations, so each larger dataset contains the smaller ones.

To measure recovery in each run, we first centre the true and fitted CDM parameters in the same way. We subtract each fitted parameter from its corresponding true value, square the difference, and add these squared differences over all distinct item pairs:

$$E = \sum_{x \neq z} \left(\widehat{u}_{xz} - u_{xz}\right)^2. \tag{27}$$

This sum is called the *squared Frobenius distance*. It is zero when all the compared parameters match, and a smaller value means better recovery. For six items, the sum contains $6 \times 5 = 30$ terms. The earlier per-entry RMSE is $\sqrt{E/30}$ for the same parameters in a single run; here we

plot the mean of E across ten runs. The dashed m^{-1} curve is a reference for comparing the trend.

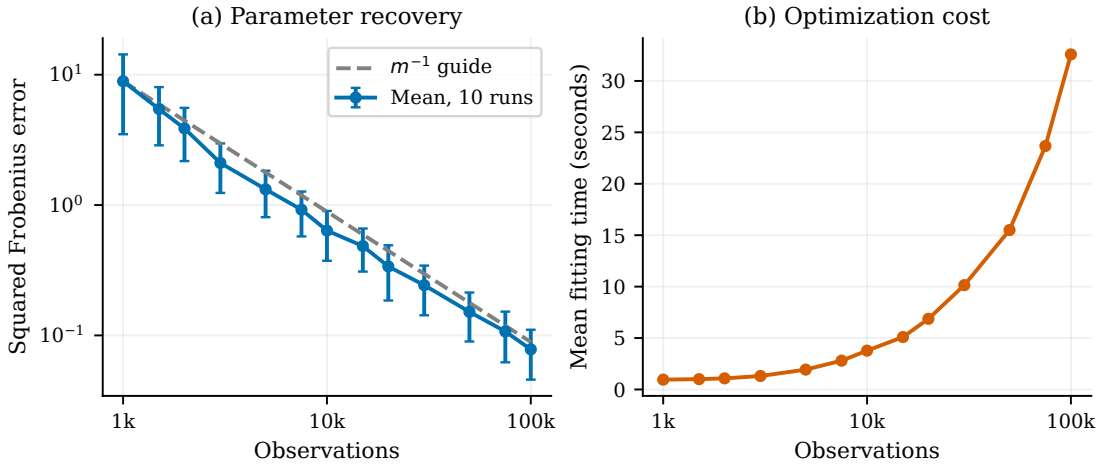


Figure 12: Six-item recovery following Seshadri et al. (a) Mean sum of squared parameter errors, with one-standard-deviation error bars over ten runs; the m^{-1} guide is anchored at the first point. (b) Mean fitting time over the same runs.

Interpretation Recovery error decreases as more observations are used, with a trend close to the m^{-1} guide over the tested range. Fitting time also increases, so better recovery here requires more computation.

4 Conclusion

This phase began with Bradley–Terry estimation and parameter recovery, and extended the study to full and low-rank context-dependent models. In the tested synthetic settings, increasing the observations improved recovery, while increasing the number of items under a fixed data budget made estimation more difficult. Fitting BTL/Luce to the tested CDM-generated choices left a predictive gap, showing the limitations of a context-independent model in those settings.

Low-rank CDM reduces the number of stored parameters, but this did not translate into the same reduction in runtime or total process memory in the present implementation. Further work will check these comparisons across independent datasets and improve optimization before drawing broader conclusions about low-rank performance.

References

[] Richard R. Batsell and John C. Polking. A new class of market share models. *Marketing Science*, 4 (3):177–198, 1985. doi: 10.1287/mksc.4.3.177.

[] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: The method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[] David R. Hunter. Mm algorithms for generalized bradley-terry models. *The Annals of Statistics*, 32 (1):384–406, 2004. doi: 10.1214/aos/1079120141.

[] Daniel McFadden. On the interpretation of the Logit model. *Working Paper*, 1975.

[] Arjun Seshadri, Alexander Peysakhovich, and Johan Ugander. Discovering context effects from raw choice data. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5660–5669. PMLR, 2019.

iography